

S03_Grupo04_Auditoria

August 3, 2026

1 Auditoría De Calidad De Datos

Integrantes: Juan Diego Serpa Ruiz, Juan Esteban Camacho Castro, Santiago Rueda Plata, Juan Pablo González Ricaurte

1.1 Introducción

1.1.1 Objetivo

Evaluar la calidad de la información en la base de datos de NovaMarket identificando anomalías, incoherencias y demás aspectos que generen una alteración en los datos. El propósito es establecer un plan de saneamiento antes de usarlos en los reportes y modelos de análisis del negocio.

1.1.2 Descripción del Dataset

```
[1]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("NovaMarket_datos_crudos (1).csv")
df.head()
```

```
[1]:  id_pedido id_cliente fecha_compra canal ciudad codigo_postal \
0    P00383    C0276  2026-01-03    App Bucaramanga      68001
1    P00403    C0167   6/12/2026    Web      Cali      76001
2    P00403    C0167   6/12/2026    Web      Cali      76001
3    P00450    C0128   07/09/2026    App bucaramanga      68001
4    P00449    C0304  2025-07-19  Tienda      Bogotá      8001
```

```
    categoria_producto  producto  precio  unidades  edad_cliente \
0          Belleza    Camiseta -1742611.0         4         -19
1          Deportes    Cafetera  1828583.0         2         23
2          Deportes    Cafetera  1828583.0         2         23
3      electronica  Set bloques   787178.0         6         69
4          Belleza  Audifonos BT   357231.0         5         67
```

```
                correo nivel_satisfaccion fecha_actualizacion_stock
0      c0276gmail.com             alto      2026-07-01
1  c0167@hotmail.com          MEDIO      2026-07-01
```

2	c0167@hotmail.com	MEDIO	2026-07-01
3	c0128@hotmail.com	MEDIO	2026-07-15
4	c0304@novamarket.co	Bajo	2026-05-20

```
[2]: print(df.describe())

print("\n", df.shape)

print("\n", df.columns)

print("\n", df.dtypes)

print("\n", df.info)
```

	codigo_postal	precio	unidades	edad_cliente
count	620.000000	5.990000e+02	620.000000	620.000000
mean	31917.129032	4.888703e+06	188.682258	48.012903
std	30339.586863	4.292512e+07	1240.652475	28.668511
min	5001.000000	-2.448455e+06	-3.000000	-20.000000
25%	8001.000000	5.914020e+05	2.000000	31.000000
50%	13001.000000	1.178666e+06	3.000000	46.000000
75%	68001.000000	1.856670e+06	5.000000	63.000000
max	76001.000000	7.730038e+08	9999.000000	200.000000

(620, 14)

```
Index(['id_pedido', 'id_cliente', 'fecha_compra', 'canal', 'ciudad',
      'codigo_postal', 'categoria_producto', 'producto', 'precio', 'unidades',
      'edad_cliente', 'correo', 'nivel_satisfaccion',
      'fecha_actualizacion_stock'],
      dtype='object')
```

id_pedido	object
id_cliente	object
fecha_compra	object
canal	object
ciudad	object
codigo_postal	int64
categoria_producto	object
producto	object
precio	float64
unidades	int64
edad_cliente	int64
correo	object
nivel_satisfaccion	object
fecha_actualizacion_stock	object
dtype:	object

```
<bound method DataFrame.info of      id_pedido id_cliente fecha_compra  canal
ciudad  codigo_postal \
0      P00383      C0276  2026-01-03      App Bucaramanga      68001
1      P00403      C0167   6/12/2026      Web      Cali      76001
2      P00403      C0167   6/12/2026      Web      Cali      76001
3      P00450      C0128  07/09/2026      App bucaramanga      68001
4      P00449      C0304  2025-07-19  Tienda      Bogotá      8001
..      ...      ...      ...      ...      ...      ...
615     P00220      C0288  2026-07-27  Tienda      NaN      68001
616     P00306      C0299  03/01/2025  Tienda Bogotá D.C.      11001
617     P00584      C0199  11/11/2025      App      BOGOTÁ      11001
618     P00205      C0323  26/03/2025      App Bucaramanga      68001
619     P00211      C0292  20/09/2025      App Bucaramanga      68001
```

```
      categoria_producto      producto      precio  unidades  edad_cliente \
0      Belleza      Camiseta -1742611.0      4      -19
1      Deportes      Cafetera  1828583.0      2      23
2      Deportes      Cafetera  1828583.0      2      23
3      electronica      Set bloques  787178.0      6      69
4      Belleza      Audífonos BT  357231.0      5      67
..      ...      ...      ...      ...      ...
615      moda      Cafetera -1864811.0      6      58
616      Electronica      Smart TV  1697118.0      1      61
617      Jugueteria      Smartphone  865190.0      6      30
618      electronica      Cafetera  757551.0      1      31
619      Belleza      Smart TV  1447761.0      6      74
```

```
      correo nivel_satisfaccion fecha_actualizacion_stock
0      c0276gmail.com      alto      2026-07-01
1      c0167@hotmail.com      MEDIO      2026-07-01
2      c0167@hotmail.com      MEDIO      2026-07-01
3      c0128@hotmail.com      MEDIO      2026-07-15
4      c0304@novamarket.co      Bajo      2026-05-20
..      ...      ...      ...
615     c0288@novamarket.co      MEDIO      2026-05-20
616      c0299@gmail.com      medio      2026-05-20
617      c0199gmail.com      Bajo      2026-06-10
618      c0323gmail.com      ALTO      2026-06-10
619      c0292@yahoo.com      alto      2026-05-20
```

```
[620 rows x 14 columns]>
```

El dataset tiene 620 filas y 14 columnas. La mayoría de los campos son de tipo texto, mientras que solo tres columnas son numéricas (codigo_postal, unidades, edad_cliente) y una es decimal (precio). A primera vista los datos presentan problemas como: valores nulos, registros duplicados, inconsistencias en el formato de textos (como diferencias entre mayúsculas y minúsculas) y datos anormales como precios, edades y unidades con valores negativos o fuera de rango.

1.2 Diagnóstico por Dimensión

1.2.1 Completitud

```
[3]: df.isnull().mean() * 100
```

```
[3]: id_pedido          0.000000
     id_cliente         0.000000
     fecha_compra       0.000000
     canal              0.000000
     ciudad             5.000000
     codigo_postal      0.000000
     categoria_producto 0.000000
     producto           0.000000
     precio             3.387097
     unidades           0.000000
     edad_cliente       0.000000
     correo             16.774194
     nivel_satisfaccion 7.419355
     fecha_actualizacion_stock 0.000000
     dtype: float64
```

```
[4]: df.isna().sum()
```

```
[4]: id_pedido          0
     id_cliente         0
     fecha_compra       0
     canal              0
     ciudad             31
     codigo_postal      0
     categoria_producto 0
     producto           0
     precio             21
     unidades           0
     edad_cliente       0
     correo             104
     nivel_satisfaccion 46
     fecha_actualizacion_stock 0
     dtype: int64
```

En cuanto a la completitud de los datos, se identificaron varios campos con datos faltantes que afectan la calidad del dataset. La variable más afectada es correo, con 104 registros nulos (16.8%), lo que representa la pérdida de información en el contacto con los clientes. Le sigue el nivel_satisfaccion, donde faltan 46 datos (7.4%), limitando la evaluación de la experiencia del usuario. Por su parte, la columna ciudad presenta 31 valores ausentes (5.0%), mientras que precio registra 21 datos faltantes (3.4%), lo cual afecta los análisis económicos en cuanto a ventas.

1.2.2 Exactitud

```
[5]: precios_negativos = (df['precio'] < 0).sum()
    unidades_negativas = (df['unidades'] < 0).sum()
    edades_negativas = (df['edad_cliente'] < 0).sum()

    percentage_precios_negativos = (df['precio'] < 0).mean() * 100
    percentage_unidades_negativas = (df['unidades'] < 0).mean() * 100
    percentage_edades_negativas = (df['edad_cliente'] < 0).mean() * 100

    print(f"Precios negativos: {precios_negativos} ({percentage_precios_negativos:.2f}%)")
    print(f"Unidades negativas: {unidades_negativas} ({percentage_unidades_negativas:.2f}%)")
    print(f"Edades negativas: {edades_negativas} ({percentage_edades_negativas:.2f}%)")
```

Precios negativos: 19 (3.06%)

Unidades negativas: 14 (2.26%)

Edades negativas: 15 (2.42%)

Ahora, respecto a la exactitud de los datos, se detectaron números negativos. En la variable precio, 19 registros (3.06%) presentan valores negativos, afectando de forma directa el cálculo de ingresos. Se observa en edad_cliente, con 15 datos anómalos (2.42%), y en unidades, donde 14 registros (2.26%) se encuentran por debajo de 0, lo que rompe la lógica total de la variable “unidades”.

1.2.3 Consistencia

```
[6]: df.groupby(['categoria_producto', 'producto']).size()
```

```
[6]: categoria_producto  producto
    Belleza             Aspiradora      6
                   Audífonos BT      5
                   Balón fútbol      4
                   Cafetera          4
                   Camiseta          6
                   ..
    moda                Muñeca         2
                   Perfume           3
                   Set bloques       3
                   Smartphone        1
                   Zapatillas        1
    Length: 218, dtype: int64
```

```
[7]: df.groupby(['ciudad', 'codigo_postal']).size()
```

```
[7]: ciudad          codigo_postal
    B/quilla        5001          1
```

	8001	22
BOGOTÁ	11001	17
	13001	1
	68001	2
	76001	1
Barranquilla	8001	28
	76001	1
Bogota	11001	20
	68001	1
Bogotá	8001	1
	11001	17
	68001	2
Bogotá D.C.	11001	17
Bucaramanga	5001	1
	8001	1
	13001	2
	68001	42
	76001	1
CALI	13001	1
	76001	35
CARTAGENA	5001	1
	8001	1
	11001	1
	13001	48
	76001	1
Cali	8001	1
	13001	1
	76001	27
Cartagena	5001	1
	8001	1
	11001	2
	13001	43
	68001	2
	76001	1
MEDELLIN	5001	25
	13001	1
Medellin	5001	23
	8001	1
	68001	1
Medellín	5001	13
	11001	1
barranquilla	8001	33
	13001	2
	68001	1
bogota	11001	14
bucaramanga	8001	2
	13001	1

	68001	55
cali	5001	1
	76001	38
medellin	5001	31
	76001	1

dtype: int64

```
[8]: df['ciudad'].unique()
```

```
[8]: array(['Bucaramanga', 'Cali', 'bucaramanga', 'Bogotá', nan, 'MEDELLIN',
        'medellin', 'BOGOTÁ', 'CARTAGENA', 'Medellin', 'CALI', 'Bogota',
        'Barranquilla', 'barranquilla', 'bogota', 'cali', 'Cartagena',
        'Bogotá D.C.', 'B/quilla', 'Medellín'], dtype=object)
```

```
[9]: df['categoria_producto'].unique()
```

```
[9]: array(['Belleza', 'Deportes', 'electronica', 'Hogar', 'deportes', 'hogar',
        'ELECTRONICA', 'belleza', 'Moda', 'juguetes', 'MODA',
        'Electronica', 'Electrónica', 'moda', 'Jugueteria', 'Juguetería',
        'HOGAR'], dtype=object)
```

```
[10]: ciudad_inconsistente = df['ciudad'].astype(str).str.strip().str.lower() !=
        df['ciudad'].astype(str)
cantidad_ciudad = ciudad_inconsistente.sum()
percentage_ciudad = ciudad_inconsistente.mean() * 100

categoria_inconsistente = df['categoria_producto'].astype(str).str.strip().str.
        lower() != df['categoria_producto'].astype(str)
total_categoria_inconsistente = categoria_inconsistente.sum()
percentage_categoria_inconsistente = categoria_inconsistente.mean() * 100

print(f"Inconsistencias_ciudad: {cantidad_ciudad} ({percentage_ciudad:.2f}%)")
print(f"Inconsistencias_categoria_producto: {total_categoria_inconsistente}
        ({percentage_categoria_inconsistente:.2f}%)")

referencia_ciudad_postal = {
    'Bogotá': '11',
    'Medellín': '05',
    'Cali': '76',
    'Barranquilla': '08',
    'Cartagena': '13'
}

def es_consistente(row):
    ciudad = row.get('ciudad', '')
    cod = str(row.get('cod_postal', ''))
    prefijo_esperado = referencia_ciudad_postal.get(ciudad, '')
```

```

    if not prefijo_esperado:
        return None
    return cod.startswith(prefijo_esperado)

df['ciudad_postal_ok'] = df.apply(es_consistente, axis=1)

inconsistentes = df[df['ciudad_postal_ok'] == False]
print(f"Inconsistencias ciudad/código postal: {len(inconsistentes)}\n
      ↳ ({len(inconsistentes)/len(df)*100:.2f}%)")

```

Inconsistencias_ciudad: 410 (66.13%)
 Inconsistencias_categoria_producto: 402 (64.84%)
 Inconsistencias ciudad/código postal: 142 (22.90%)

```

[11]: print("Productos:", list(df['producto'].unique()))
      print("\nCategorías Productos:", list(df['categoria_producto'].unique()))

referencia_producto_categoria = {
    'Camiseta': 'moda',
    'Cafetera': 'hogar',
    'Set bloques': 'juguetería',
    'Audífonos BT': 'electrónica',
    'Licuadora': 'hogar',
    'Perfume': 'belleza',
    'Muñeca': 'juguetería',
    'Smart TV': 'electrónica',
    'Zapatillas': 'moda',
    'Crema facial': 'belleza',
    'Smartphone': 'electrónica',
    'Mancuernas': 'deportes',
    'Balón fútbol': 'deportes',
    'Aspiradora': 'hogar'
}

def es_consistente_categoria(row):
    producto = row.get('producto', '')
    categoria_actual = str(row.get('categoria_producto', '')).strip().lower()

    categoria_esperada = referencia_producto_categoria.get(producto, '')
    if not categoria_esperada:
        return None

    return categoria_actual == categoria_esperada

df['producto_categoria_ok'] = df.apply(es_consistente_categoria, axis=1)

```



```

inconsistentes = df[df['producto_categoria_ok'] == False]
print(f"\nProductos en categoría incorrecta: {len(inconsistentes)}\n")
    ↳ ({len(inconsistentes)/len(df)*100:.2f}%)")

```

Productos: ['Camiseta', 'Cafetera', 'Set bloques', 'Audífonos BT', 'Licuadora', 'Perfume', 'Muñeca', 'Smart TV', 'Zapatillas', 'Crema facial', 'Smartphone', 'Mancuernas', 'Balón fútbol', 'Aspiradora']

Categorías Productos: ['Belleza', 'Deportes', 'electronica', 'Hogar', 'deportes', 'hogar', 'ELECTRONICA', 'belleza', 'Moda', 'juguetes', 'MODA', 'Electronica', 'Electrónica', 'moda', 'Jugueteria', 'Juguetería', 'HOGAR']

Productos en categoría incorrecta: 551 (88.87%)

En cuanto a la consistencia de los datos, encontramos varios fallos al momento de agrupar y organizar la información. El primer error está en la asignación de categorías, donde hay muchos productos guardados en la sección equivocada (como una aspiradora en belleza). También hay un problema en ciudad y categoria_producto, ya que la misma palabra está escrita de formas distintas, lo que hace que el sistema las tome como cosas diferentes. Por último, hay un error entre la ciudad y el codigo_postal, registrando códigos postales de una ciudad en otra.

1.2.4 Validez

```

[12]: correos_inexactos = df['correo'].dropna().map(lambda x: '@' not in x or '.' not in x).sum()
    porcentaje_correos_inexactos = df['correo'].dropna().map(lambda x: '@' not in x or '.' not in x).mean() * 100

print(f"Correos inexactos: {correos_inexactos} ({porcentaje_correos_inexactos:.2f}%)")

```

Correos inexactos: 51 (9.88%)

```

[13]: fechas_invalidas = pd.to_datetime(df['fecha_compra'], errors='coerce').isnull()
fechas_invalidas_reales = fechas_invalidas & df['fecha_compra'].notna()
print(f"Fechas con formato inválido: {fechas_invalidas_reales.sum()}\n")
    ↳ ({fechas_invalidas_reales.sum()/len(df)*100:.2f}%)")

```

Fechas con formato inválido: 401 (64.68%)

En cuanto a la validez de los datos, encontramos registros con formatos incorrectos que impiden procesar la información. Lo primero es que en la columna fecha_compra, hay 401 fechas con un formato inválido que no se pueden leer correctamente para saber cuándo se realizó dicha venta. Además, encontramos 51 correos electrónicos que están mal escritos porque les falta el arroba (@) o el punto (.).

1.2.5 Unicidad

```
[14]: duplicados_clave = df.duplicated(subset=['id_pedido']).sum()
print(f"Pedidos con misma id_pedido: {duplicados_clave} ({duplicados_clave/
↳len(df)*100:.2f}%)")
```

Pedidos con misma id_pedido: 20 (3.23%)

En cuanto a la unicidad, encontramos 20 registros duplicados (3.23%), donde se repite exactamente el mismo número de pedido (id_pedido) y toda la información de la fila. Esto significa que hay compras registradas dos veces de forma idéntica.

1.2.6 Oportunidad

```
[15]: fechas_dt = pd.to_datetime(df['fecha_compra'], errors='coerce')

fecha_min = fechas_dt.min()
fecha_max = fechas_dt.max()
print(f"Período cubierto por el dataset: {fecha_min.date()} -> {fecha_max.
↳date()}")
print(f"Duración: {(fecha_max - fecha_min).days} días")

hoy = pd.Timestamp('2026-08-03')
dias_desde_ultimo = (hoy - fecha_max).days
print(f"Días desde el último registro: {dias_desde_ultimo}")

fechas_futuras = fechas_dt > hoy

cant_futuras = fechas_futuras.sum()
pct_futuras = (cant_futuras / len(df)) * 100

print(f"Registros con fechas futuras (> {hoy.date()}): {cant_futuras}
↳({pct_futuras:.2f}%)")
```

Período cubierto por el dataset: 2025-01-08 -> 2027-05-20

Duración: 862 días

Días desde el último registro: -290

Registros con fechas futuras (> 2026-08-03): 46 (7.42%)

Al analizar el rango de fechas, encontramos que el dataset va desde el 8 de enero de 2025 hasta el 20 de mayo de 2027, sumando un total de 862 días. El dato que mas resalta es que tenemos 46 registros (7.42%) con fechas futuras, posteriores al 3 de agosto de 2026. Incluso el registro más lejano supera la fecha actual por 290 días.

1.3 Catálogo de Problemas

Problema	Dimensión	Afectadas	Registros	Impacto
Datos faltantes / nulos	Compleitud	correo, nivel_satisfaccion, ciudad, precio	correo: 104 (16.77%) nivel_satisfaccion: 46 (7.42%) ciudad: 31 (5.00%) precio: 21 (3.39%)	Alto
Valores numéricos imposibles	Exactitud	precio, edad_cliente, unidades	precio: 19 (3.06%) edad_cliente: 15 (2.42%) unidades: 14 (2.26%)	Alto
Inconsistencia por formato	Consistencia	ciudad, categoria_producto	ciudad: 410 (66.13%) categoria_producto: 402 (64.84%)	Medio
Discordancia geográfica	Consistencia	ciudad, codigo_postal	ciudad / codigo_postal: 142 (22.90%)	Medio
Asignación errónea de categoría	Consistencia	producto, categoria_producto	producto / categoria_producto: 551 (88.87%)	Medio
Formato de correo y fecha inválidos	Validez	fecha_compra, correo	fecha_compra: 401 (64.68%) correo: 51 (9.88%)	Alto
Filas o claves duplicadas	Unicidad	id_pedido, Filas completas	id_pedido: 20 (3.23%) Filas exactas: 20 (3.23%)	Medio
Fechas futuras respecto al análisis	Oportunidad	fecha_compra	fecha_compra: 46 (7.42%)	Bajo

1.4 Priorización

1. Correos y fechas con mal formato: Hay muchísimas fechas de compra invalidas (64.68%), así que no podemos saber cuando se vendió qué. Los correos mal escritos tambien hacen que a los clientes no le lleguen las facturaciones o demás notificaciones.

2. Datos que faltan/nulos: Falta el 3.39% de precios de venta y el 16.77% de correos, lo que hace que no le lleguen la facturacion al cliente o notificaciones y ademas al hacer los calculos del total de ventas da mal el resultado.

3. Números negativos o imposibles: No existen precios ni cantidades vendidas en negativo. Estos números dañan de inmediato las sumas y las ganancias totales.

4. Productos en la categoría equivocada: Pasa en casi todo el dataset (88.87%). La venta

se hizo, pero si metes una Aspiradora en la sección de Belleza, las cuentas por área quedan mal hechas y puede generar analisis erroneos como decir que las mayores ventas fueron en belleza pero los productos que se vendieron son de Hogar.

5. Mismo dato escrito de varias formas: Pasa en más del 60% de las ciudades y categorías. Si el sistema ve “Bogotá” y “bogota”, piensa que son dos ciudades distintas y al hacer informes de ventas por ciudades por ejemplo, vamos a tener 19 ciudades cuando en realidad son 5.

6. Ciudades con código postal equivocado: No daña las cuentas del dinero o procesos de reportes, pero sí se realiza un envío puede terminar en una ciudad erronea.

7. Pedidos repetidos: Hay un 3.23% de ventas duplicadas, lo que altera la realidad de las ventas haciendo creer que se vendió mas de la realidad.

8. Fechas del futuro: Hay compras registradas con fechas que aún no han pasado (como mayo de 2027). Toca analizar que pudo haber ocurrido y decidir si se modifica o se eliminan dichas filas.

1.5 Recomendaciones

Para las fechas, convertiremos fecha_compra a formato válido, dejando nulas las ilegibles y corrigiendo las 46 fechas futuras. Para los correos faltantes o inválidos, se asignará un valor como ‘sin_correo’.

En la parte numérica, pasaremos a positivo los precios, edades y unidades negativas. Para los precios faltantes, se calculará el valor según el tipo de producto.

Limpiaremos los textos de ciudad y categoria_producto quitando espacios y pasándolos a minúsculas para unificar los nombres. Además, ajustaremos los productos en la categoría correcta y los códigos postales según su ciudad correspondiente.

Por último, eliminaremos los duplicados para borrar las 20 filas repetidas y dejar solo una por pedido.